HW 2.2 | Parallelizing a Particle Simulation

Benjamin Krohn-Hansen, Brandon Ton, Jaeyoon Jung | CMPSCI 267 | S25

INTRODUCTION

This homework assignment is an extension of HW 2.1, switching from a shared-memory (OpenMP) implementation to a distributed-memory implementation using MPI. In the simulation, particles have repulsive forces at short range. The code has been optimized to have an approximated O(n) complexity by taking advantage of low particle density. We implement a 1D domain composition, a 2D domain decomposition, ghost particle exchange, and particle migration to handle particles that move between processor subdomains. Particle communication is handled with two different communication routines: collective and point-to-point. Additionally, variable vector sizes are handled by communicating vector sizes prior to sending vectors themselves.

IMPLEMENTATION APPROACH

To ensure scalability and efficiency with several MPI processes, the parallel code has been designed around several data structures. Regarding local domain and particle storage, each MPI process is assigned a subdomain of the global simulation space by implementing a 1D grid decomposition. Grid dimensions are computed as size divided by the number of ranks. We define a vector 'local_parts', which are particles that lie within a process' subdomain.

Regarding communication across process boundaries, ghost particle vectors are defined for different types of neighboring ranks above and below the current rank. These allow each process to exchange data with only immediate neighbors, which helps avoid global communication and keeps the communication local.

Our implementation focuses on reducing the initial $O(n^2)$ cost of force calculation by applying a local binning or tiling strategy with MPI communication techniques. Each MPI process is assigned a rectangular row of the overall simulation, which is further subdivided into smaller tiles with dimensions roughly equal to the interaction cutoff. We utilize a vector of vectors to store particles within each tile. Within simulate_one_step(), we assign each particle to exactly one tile, based on the x and y coordinates relative to the local subdomain. This local binning ensures that particles are grouped spatially. We limit the calculation of forces to only neighboring tiles, which avoids checking forces against all $O(n)$ local particles. Due to the bin size being approximately the cutoff, any particles more than one tile away will be excluded by the cutoff distance. This drastically reduces the pairwise checks necessary, decreasing the initial $O(n^2)$ cost.

We also implement non-blocking send and receive (MPI_Isend and MPI_Irecv) along with MPI_Waitall calls to minimize the time spent waiting idly. Our communication phases include ghost particle count exchange, ghost particle data exchange, migrated particle count exchange, and migrated particle data exchange. By implementing non-blocking communications and MPI_Waitall at critical points, we allow processes such as force calculations to continue without waiting for data from all ranks.

COMMUNICATION (Distributed memory implementation)

The MPI implementation relies on explicit message passing to handle data exchanges between process subdomains. One key communication aspect includes the ghost particle exchange section. Each process identifies particles near its subdomain boundaries (which is defined within one cutoff distance) that need to be sent as ghost particles to neighboring processes.

Particle migration is yet another instance of the communication aspect that was implemented. After updating particle positions, some particles move out of a process' subdomain. These particles migrating are identified and subsequently removed from the local vector. Afterwards, they are exchanged with neighboring processes using nonblocking calls. During this process, the counts and particle data are exchanged, and received particles are merged into the local set.

Additionally, in the output section, rank 0 gathers the complete particle data from all processes using MPI_Gather and MPI_Gatherv, then sorts the particles by their global ID. The aforementioned communication strategies work to minimize the amount of data by explicitly restricting communication to neighboring processes. This is further applied by performing migration in a separate phase than ghost exchanges. MPI_Barrier assists in synchronizing processes between various communication phases.

DESIGN

1D Decomposition:

Other communication alternatives were considered before finally landing on a combination of synchronization and point-to-point communication techniques. Initially, MPI collectives such as MPI_Alltoallv were considered for the ghost exchange section. However, upon analysis, these collectives were deemed too computationally greedy, as we are only dealing with neighbor processes in this assignment. As a result, we settled on point-to-point communication utilizing MPI_Sendrecv in both particle exchange and particle migration. This technique ensures that each process communicates only with their immediate neighbors. We also considered using Blocking Send Receive, but since ranks would have sent two sets of data (above and below) and received two sets of data, we decided to perform these communications simultaneously using nonblocking communication and Waitall.

To ensure synchronization, barriers were implemented to ensure that each process completed a communication phase before moving to the next step in the simulation. This was done via an MPI_Barrier at the end of each step.

2D Domain Decomposition:

The particle simulation utilizes a 2D grid-based domain decomposition strategy, where the simulation space is divided into square grid cells managed by individual MPI processes. Each process is responsible for a specific grid cell and communicates with its eight possible neighboring cells (LEFT, RIGHT, ABOVE, BELOW, TOP LEFT, TOP RIGHT, BOTTOM LEFT, and BOTTOM RIGHT) using nonblocking MPI communication (MPI_Isend, MPI_Irecv) combined with MPI_Waitall. The communication design consists of four key steps: 1) exchanging ghost particle counts, 2)

transmitting actual ghost particle data, 3) sending the count of particles crossing grid boundaries, and 4) transferring the particle data itself. The use of direction tags and direction-based communication ensures precise message matching and prevents deadlock, which is crucial in a 2D grid where particles may move in any direction.

To efficiently handle particle data, the design employs a std::vector<particle_t> data structure, with separate vectors for each communication direction. This approach maintains memory contiguity, enabling direct memory access for MPI operations without serialization. MPI_Barrier placements ensure synchronization across processes, particularly important in a 2D grid where communication dependencies can be more complex than in a 1D grid. Additionally, debugging outputs and request status checks (MPI_Test, MPI_Waitall) provide enhanced visibility into communication flow, helping identify and resolve potential issues. This robust design supports scalability and high performance, making it suitable for large-scale simulations that involve complex particle interactions across a 2D domain.

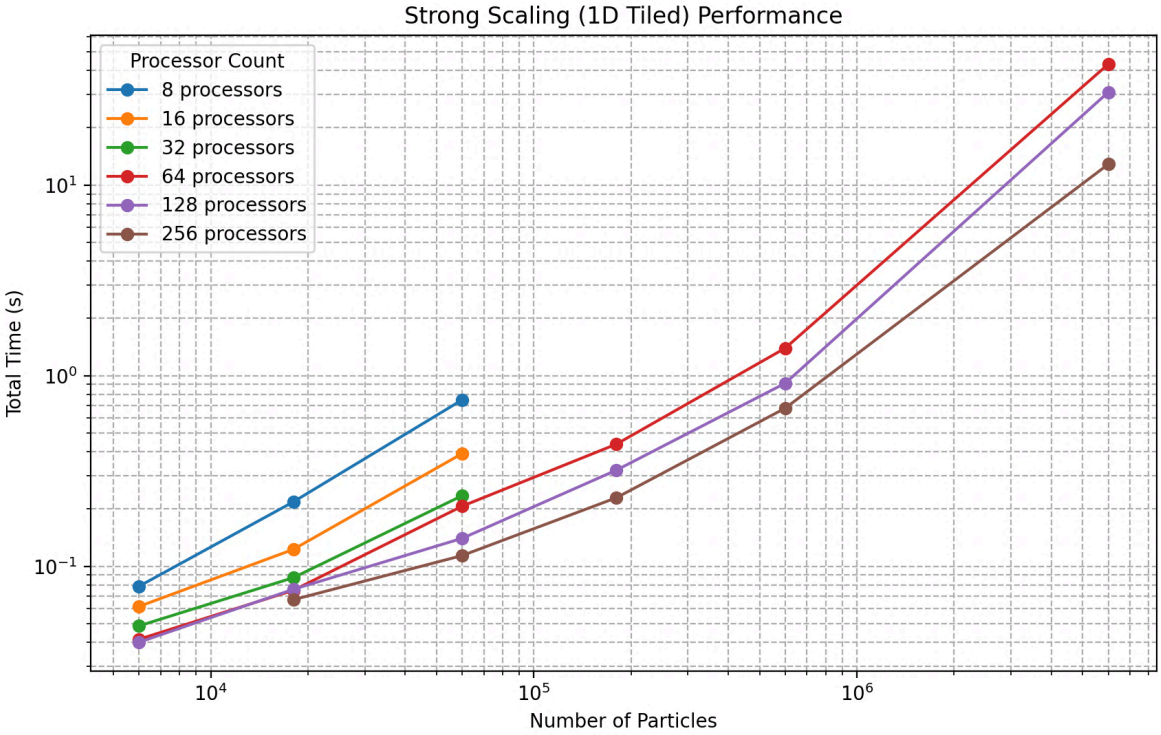
Limitations and Improvement Measures

Unfortunately, our 2D grid decomposition couldn't give the result due to deadlock. The primary limitation of the 2D grid decomposition approach in the particle simulation was the increased complexity and communication overhead compared to a 1D grid. Unlike 1D grids, where communication occurs only with adjacent processes on either side, a 2D grid requires communication with up to eight neighboring cells. This significantly amplifies the risk of communication mismatches and deadlocks, especially when message tags, counts, or directions do not align perfectly. The intricate boundary conditions and the potential for particles to cross multiple grid boundaries simultaneously add further complexity, often causing the code to stop if any communication step is not perfectly synchronized.

To improve this and eventually make the code work, several measures could be implemented. Introducing more granular debugging tools, such as rank-specific logs with detailed message tracing, would help identify precisely where communication mismatches occur. Also, adopting a halo-based exchange pattern with overlapping grid boundaries could minimize the need for complex diagonal communications. Finally, dynamic load balancing or adaptive grid resizing might help avoiding scenarios where certain ranks are overwhelmed with particles, leading to communication bottlenecks.

SCALING AND RUNTIME

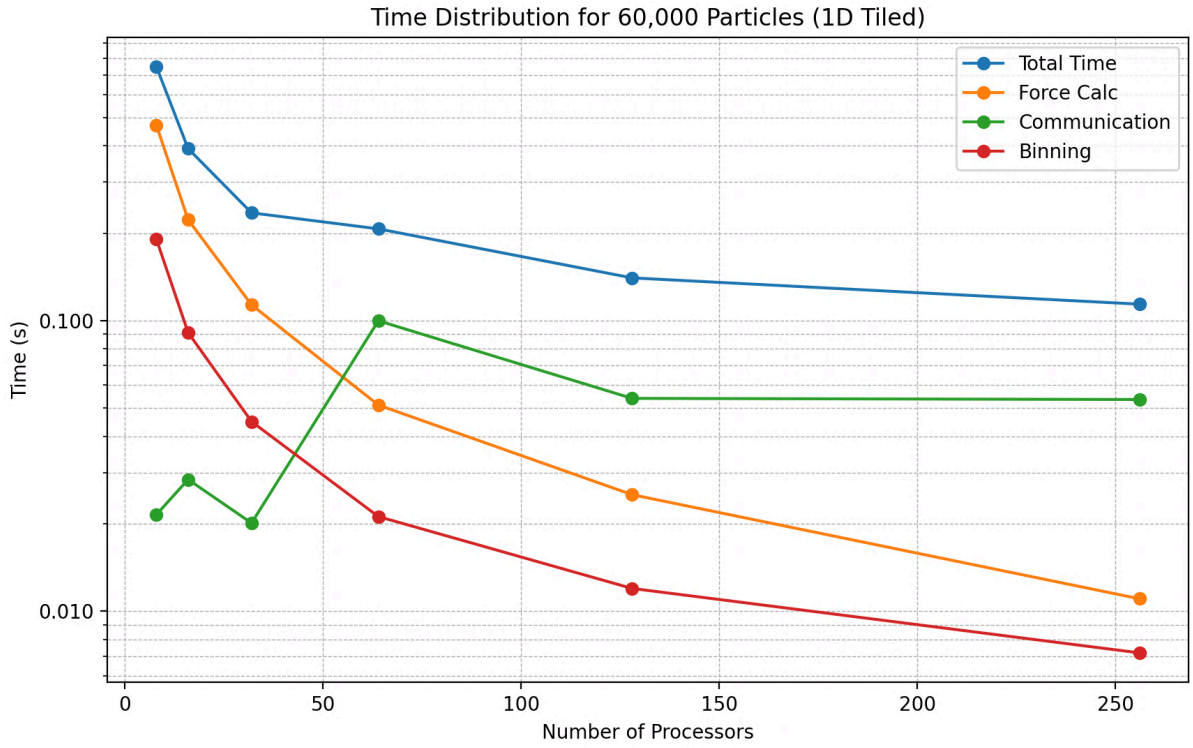
STRONG SCALING PLOT (1D strong scaling)



Regarding strong-scaling experiments, we kept the total number of particles constant while changing the number of MPI processes. The general trend is that as the number of particles increases, each line (representing a fixed number of processors) has a positive slope, indicating that total execution time grows with larger particle sizes. This pattern is to be expected, as increasing particles require more force calculations and ghost particle communication.

Higher processor counts (128 and 256) generally yielded much lower execution times than smaller numbers of processors, suggesting that the simulation does benefit from parallelization with large enough particle numbers. At smaller particle counts, the graphs are generally close together, demonstrating that more processors does not drastically reduce time, which can be attributed to overhead costs of communication of ghost particles and synchronization. At larger particle counts, increasing processors has a more drastic effect in reduction of total time, showing that perhaps computation is better amortized across ranks.

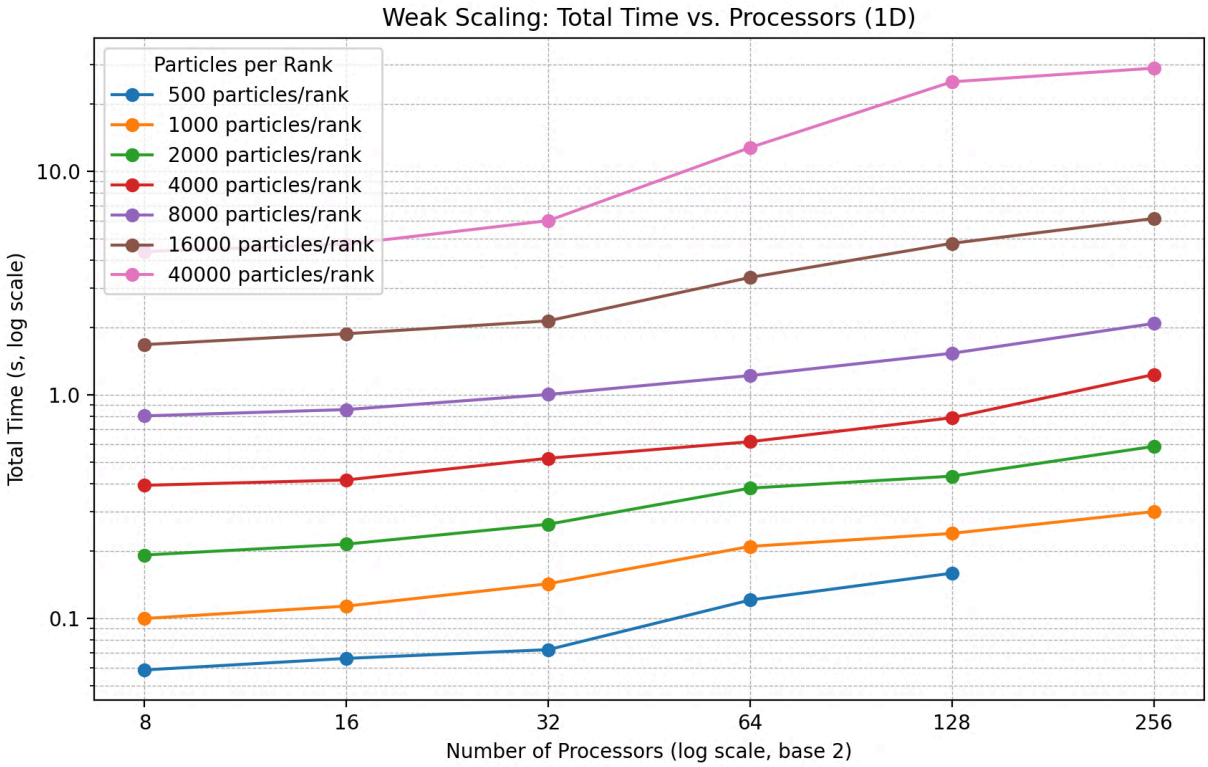
WHERE DID THE TIME GO?



The discussion of runtime can be categorized into a few sections: total time, time spent in force calculations, communication time, and binning time. An overall trend we notice is that total time decreases with more processors. This indicates that generally, parallelization is succeeding in reducing the overall runtime.

Both force calculations and particle binning portray the same trend, with a scaling close to 1/p. For a lower number of processors, force calculations and binning are the main contributors to overall run time. However, as the number of ranks increases (64, 1228, 256), each rank has fewer particles to sort into bins and perform force calculations, thus dramatically decreasing time spent in each section. The trend for communication time is a little bit more nuanced. At lower ranks, communication time quickly becomes the dominant factor in run time due to increasing overhead introduced by managing communication between more ranks. However, beyond 64 ranks, communication time decreases due to less particles existing per rank, resulting in less information being available for send and receive.

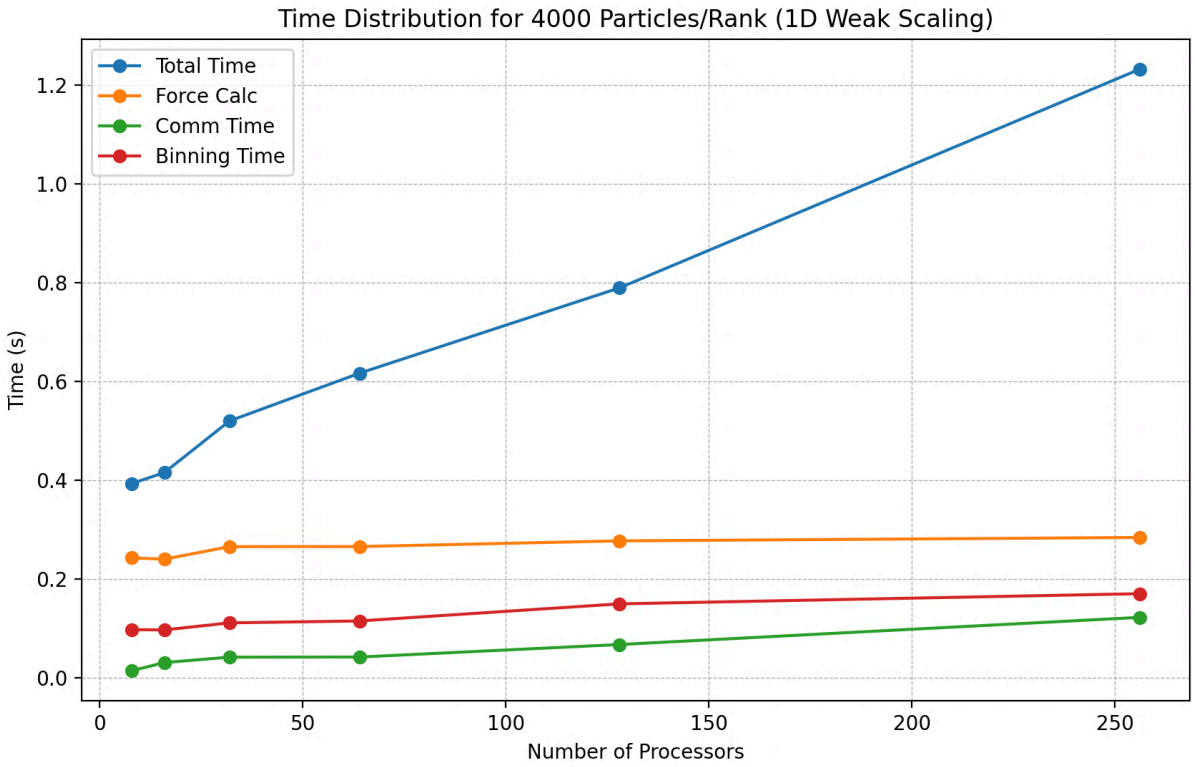
WEAK SCALING PLOT (1D tiling)



In the weak-scaling experiment, we increased the number of particles proportionally with the number of processes. An ideal weak scaling graph would show that simulation time per process remains relatively constant. Overall, the trend is that total execution time slightly increases with the number of processors, which is expected because communication overhead and synchronization between ranks can increase with an increasing number of processors. While our weak-scaling lines are not perfectly flat, the weak-scaling for fewer particles per rank is decent. However, as we increase particles per rank, we start to notice deviation from perfect weak scaling. Although we have more local computation to amortize overhead, eventually we do run into issues with communication and synchronization costs becoming more significant.

Directly comparing smaller and larger particles per rank, smaller particles/rank has the shortest runtime at each processor count. This makes sense, since each process has the fewest local particles. Likewise, larger particles per rank is expected to have higher total execution time for the same processor since each process is performing more local computations, and exchanging more ghost particles.

WHERE DID THE TIME GO?



As we increase the number of processors while maintaining constant particles, ideally we would expect the time spent in force calculation, communication, and binning to remain constant. The results of the plot suggest that parallelization of force calculation is close to ideal, with some room for improvement for communication time and binning time. Force calculation time remains constant at around 0.25 seconds for all numbers of processors. Communication time grows slightly compared to both the force calculation and binning time. As we increase the ranks, the volume of MPI messages may increase. Each rank may experience more boundary exchanges among ranks, which would lead to overhead accumulating and increasing total time.

Ultimately, there are slight deviations from ideal for both communication and binning times. A rise in both of these as ranks increase indicates that either data exchange or subdomain management is less than ideal. Most of the extra time contributing to total time increase may be a result of the waiting period in MPI_Barrier or MPI_Waitall, which isn't explicitly captured in our time breakdown. Overall, we see that the communication, force calculation, and particle binning are well parallelized, as their runtimes increase minimally. We attribute the increase in total time to serial parts of the program – particularly simulation initialization.

SUGGESTIONS FOR IMPROVEMENTS:

There are some further optimizations that we could consider. Currently, we send the entire particle_t structures (including velocity and acceleration, and id) for both ghost exchange and

migrations. We could consider optimizing communication by only sending selective data when possible, decreasing the volume of communication. We could also potentially see speedup by overlapping communication with local computation. For example, while receiving ghost particle data, we could begin computing force contributions not contingent on ghost data, but this would be difficult to implement non-blocking calls.

INDIVIDUAL CONTRIBUTIONS:

Ben: Initial 1D parallelization. 1D parallelization with tiling. Performance analysis + communication vs. computation time breakdown. Editing write-up.

Brandon: 1D Parallel implementation, 1D tiling, Parallel code performance write-up and analysis, distributed memory implementation (communication), design write-up

Jae: 2D grid implementation and debugging, strong and weak scaling graph of 1D grid, write-up